ToolBridge — Enterprise Model Context Protocol (MCP) Gateway

Technical Architectural Specification, Security Engineering & Portfolio Documentation

Architect: Bhavesh Bargat | **Stack:** Python 3.12 • MCP SDK v2.3.0 • FastAPI • Streamlit • SQLite (WAL) • PostgreSQL | **Tests:** 180 Passed (100% Green, 0 Skipped)

1. Project Overview

ToolBridge is a centralized, secure **Enterprise Model Context Protocol (MCP) Gateway** engineered to aggregate, govern, and multiplex disparate upstream data servers into a unified, namespaced downstream interface for autonomous AI applications (e.g., Claude Desktop, Cursor IDE, LangChain agents). Built strictly on the official Python MCP SDK (v2.3.0), ToolBridge operates as the foundational security and policy enforcement tier in the AI tool-calling lifecycle.

The Problem It Solves: Autonomous LLMs equipped with tool-calling capabilities cannot be safely granted unrestricted access to enterprise filesystems, databases, or cloud APIs. LLMs are susceptible to prompt injection, hallucinated arguments, directory traversal attacks (../../etc/passwd), and destructive SQL mutations (DROP TABLE). Furthermore, desktop AI clients cannot natively multiplex multiple stdio subprocesses without connection leaks, protocol framing corruption, and an absence of unified audit logging.

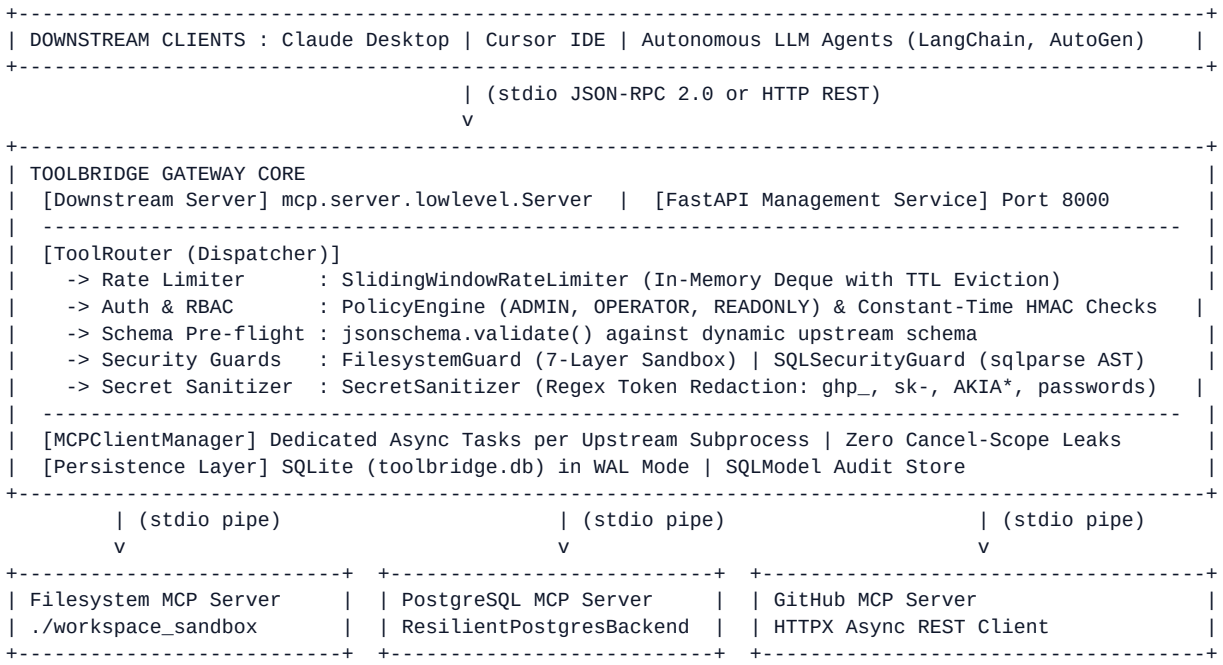
Real-World Utility & Target Users: Designed for enterprise platform architects, AI engineers, and security teams deploying LLM agents in production. ToolBridge acts as the 'API Management and Reverse Proxy' layer for AI agents, enforcing zero-trust guardrails, schema pre-flight validation, and compliance-grade audit logging without requiring any modifications to upstream services or downstream foundation models.

2. Technology Stack & Architectural Rationale

Category	Technology	Version	Implementation Purpose & Architectural Rationale
Core Language	Python	3.12+	Asynchronous I/O (asyncio), structural pattern matching, robust type annotations, and native subprocess streams.
Protocol Core	MCP Python SDK (mcp, mcp-types)	v2.3.0	Official protocol library. Implements low-level mcp.server.lowlevel.Server for dynamic multiplexing and mcp.client.stdio.stdio_client for upstream process lifecycle management.
Web API Core	FastAPI & Uvicorn	0.115 / 0.30	REST management API, OpenAPI Swagger docs (/docs), health probes, and programmatic tool invocation endpoints.
Validation	Pydantic v2 & jsonschema	2.10 / 4.23	Strict data models and pre-flight argument validation against dynamic upstream JSON schemas before process dispatch.
SQL Parsing	sqlparse	0.5.0	Abstract Syntax Tree (AST) tokenization to enforce single SELECT queries and block data-modifying CTEs/injections.
Database / ORM	SQLModel & SQLite / asyncpg	0.0.22 / 0.30	Async persistence for audit logs in SQLite (WAL mode). Controlled PostgreSQL connection pool with automated SQLite fallback.
Observability	Streamlit & Tailwind SPA	1.40 / ES6	Real-time telemetry dashboard (port 8501) and embedded single-page Web Control Center with 1-click attack simulation.
Testing & CI	Pytest, RESPX, Ruff	8.3 / 0.23 / 0.8	180 automated tests (unit, security penetration, integration) and GitHub Actions CI matrix for Ubuntu & Windows.

3. System Architecture & Component Interaction

ToolBridge establishes a bidirectional architectural boundary. It functions as a single downstream MCP Server to client applications (such as Claude Desktop or autonomous agents) over **stdio JSON-RPC 2.0** or **HTTP REST**, while acting as an asynchronous MCP Client Manager to isolated upstream worker processes.



4. End-to-End Execution Workflow

- **1. Wire Interception:** Downstream AI client dispatches a JSON-RPC 2.0 tools/call frame across standard input (or POST to /api/v1/tools/invoke).
- **2. Identity & Rate Quota:** Caller identity is authenticated via Bearer token or transport defaults (stdio defaults safely to readonly). SlidingWindowRateLimiter verifies request frequency within the sliding window.
- **3. Routing & Namespacing:** The router splits __ on the first __ delimiter, locates the target upstream in ServerRegistry, and asserts server health.
- **4. Policy & Pre-Flight Validation:** PolicyEngine checks caller role permissions against the tool sensitivity tier (SAFE vs SENSITIVE). jsonschema verifies argument types before process invocation.
- **5. Upstream Process Execution:** Request is routed to the dedicated asyncio worker task managing the target subprocess. Operations pass through deep guardrails (e.g., FilesystemGuard checks canonical containment; SQLSecurityGuard validates single SELECT AST).
- **6. Secret Redaction:** Upstream outputs pass through SecretSanitizer, recursively scrubbing tokens (ghp_, sk-, AKIA), passwords, and connection strings to ***REDACTED***.
- **7. Audit Logging & Protocol Return:** Execution metadata (duration, caller, status, sanitized arguments) is asynchronously committed to SQLite, and standard MCP CallToolResult is returned on stdout.

5. Core Features & Security Guardrails

A. 7-Layer Filesystem Sandbox (FilesystemGuard)

Protects the host operating system against path traversal, symlink escapes, and Windows device lockups:
(1) **Null-Byte Defense:** Rejects \0 to stop C-level string truncation; (2) **Relative Path Enforcement:** Rejects leading slashes, backslashes, and drive letters; (3) **Multi-Dot Neutralization:** Blocks .../ traversal bypasses; (4) **Canonical Resolution:** Resolves all relative components via Path.resolve(); (5) **Sandbox Containment:** Enforces resolved.relative_to(sandbox_root); (6) **Symlink Defense:** Asserts symlink targets resolve inside the sandbox; (7) **Windows DOS Device Defense:** Blocks reserved basenames (CON.txt, AUX.log, NUL) and Alternate Data Streams (:); (8) **Atomic Writes:** Writes to tempfile.mkstemp followed by atomic os.replace.

B. Abstract Syntax Tree (AST) SQL Guard (SQLSecurityGuard)

Replaces fragile regex matching with full AST tokenization using sqlparse:
• **Single Statement Enforcement:** Immediately rejects stacked queries (e.g., SELECT 1; DROP TABLE users;);
• **Command Whitelist:** Root statement type must be strictly SELECT;
• **Mutation CTE Defense:** Traverses syntax trees to block data-modifying Common Table Expressions (e.g., WITH x AS (DELETE ... RETURNING *) SELECT ...);
• **Clause & Function Blacklist:** Blocks SELECT ... INTO, pg_sleep, pg_terminate_backend, dblink, and pg_read_file;
• **Engine Isolation:** PostgreSQL connection pool executes under SET SESSION CHARACTERISTICS AS TRANSACTION READ ONLY.

C. Governance, RBAC & Secret Scrubbing

• **Two-Tier RBAC:** Classifies tools as SAFE (read-only) or SENSITIVE (mutating). Maps to roles ADMIN (full access), OPERATOR (safe + file writes), and READONLY (safe tools only, sensitive invocations rejected with HTTP 403).
• **Constant-Time Authentication:** Validates 32+ char API keys with hmac.compare_digest without early exits, eliminating timing attack vectors.
• **Automated Secret Redaction:** Sanitizes credentials and regex tokens (ghp_, sk-, AKIA*, DSN passwords) from audit logs and client context windows.

6. AI Ecosystem Role & Agent Integration

Architectural Distinction: ToolBridge is not a machine learning model trainer; it is an **Agent Tool Gateway and Security Reverse Proxy** operating in the inference execution path of Foundation Models (Claude 3.5 Sonnet, GPT-4o, Gemini 1.5 Pro). Autonomous models generate JSON-RPC tool-calling payloads; ToolBridge intercepts, validates, and executes them on behalf of the AI.
• **Dynamic Schema Export:** Exposes JSON schemas conforming to the MCP specification to downstream models.
• **Context Window Protection:** Scrubbing passwords and tokens prevents accidental data exfiltration into LLM context.
• **Non-Fatal Error Semantics:** Policy violations and guard denials return structured tool results with is_error=True rather than crashing the protocol, allowing the agent to self-correct its query in the next turn.

7. Complete Project Folder Structure

Directory / File	Architectural Responsibility
src/toolbridge/server/gateway.py	Downstream stdio MCP Gateway server. Binds stdin/stdout; routes logs exclusively to stderr to maintain JSON-RPC wire purity.
src/toolbridge/router/dispatcher.py	Central ToolRouter. Handles namespacing (server__tool), schema validation, RBAC checks, and execution dispatch.
src/toolbridge/client/manager.py	Subprocess lifecycle manager. Solves AnyIO cancel scope race conditions by giving each upstream a dedicated asyncio worker task.
src/toolbridge/client/discovery.py	ToolDiscoveryEngine. Paginates tools/list using cursor exhaustion and registers schemas into ServerRegistry.
src/toolbridge/security/guards/	Defensive security modules: filesystem.py (7-layer sandbox) and sql.py (sqlparse AST inspector).
src/toolbridge/security/policy.py	PolicyEngine enforcing SAFE vs SENSITIVE tool tiers across ADMIN, OPERATOR, and READONLY caller roles.
src/toolbridge/api/app.py	FastAPI management service factory, lifecycle hooks, static web SPA mounting, and development key injection.
src/toolbridge/dashboard/app.py	Streamlit control plane dashboard with real-time server health cards, telemetry charts, and Admin Key gate.
src/toolbridge/upstreams/	Managed upstream servers: filesystem/, postgres/ (with SQLite fallback), and github/ (hermetic mockable client).
tests/ (180 tests)	Comprehensive automated test suite: tests/security/, tests/integration/, and tests/unit/.

8. Testing, Quality & Performance Verification

Automated Test Suite Execution (pytest 9.2.0):
- **Total Tests Collected:** 180 | **Passed:** 180 (100% Green) | **Failed:** 0 | **Skipped:** 0 | **Suite Execution Time:** 66.34 seconds.
- **Static Analysis (ruff 0.8.0):** All checks passed with zero errors or warnings.
- **Diagnostics Check (run_demo.py check):** 3/3 components verified (Filesystem, Resilient Database, GitHub API).
- **Subprocess Integrity:** 20-iteration connection stress testing verified zero orphaned child processes or leaked PIDs.
- **Hermetic GitHub Testing:** Tested using respx with zero physical network socket calls.

9. Professional Resume-Ready Content

Project Summary (2–3 Lines):

ToolBridge – Enterprise Model Context Protocol (MCP) Gateway: Architected an enterprise-grade, asynchronous MCP gateway in Python (FastAPI, MCP SDK v2.3, SQLModel) that aggregates and multiplexes upstream data servers into a unified tool catalog for autonomous AI agents (Claude Desktop, Cursor). Engineered zero-trust security controls including an AST-based SQL query guard, a 7-layer filesystem sandbox, cryptographic RBAC, and automated secret redaction, backed by a 180-test automated verification suite.

Technical Resume Bullet Points:

- **Designed and implemented** an enterprise Model Context Protocol (MCP) gateway in Python 3.12, multiplexing multiple upstream servers (PostgreSQL, Filesystem, GitHub) over stdio and REST interfaces for downstream LLM agents.
- **Engineered zero-trust security guardrails**, including an AST-tokenized SQL inspection engine (sqlparse) preventing injection/mutation and a 7-layer canonical filesystem sandbox blocking directory traversal, Windows device exploits, and symlink escapes.
- **Architected robust asynchronous subprocess lifecycles** with asyncio and mcp SDK v2.3, eliminating AnyIO cancel scope race conditions and guaranteeing stdio JSON-RPC wire purity by isolating all logs to stderr.

- **Developed centralized governance and observability controls**, incorporating constant-time HMAC API authentication, 3-tier RBAC (ADMIN, OPERATOR, READONLY), in-memory sliding-window rate limiting, and automated CI pipelines with 180 passing integration/security tests.

ATS Keywords & Core Competencies:

Model Context Protocol (MCP), LLM Agent Tooling, API Gateway, Reverse Proxy, Asynchronous Concurrency, Python 3.12, FastAPI, Claude Desktop Integration, Cursor IDE, AST Parsing, SQL Injection Defense, Filesystem Sandboxing, Zero-Trust Architecture, Role-Based Access Control (RBAC), Rate Limiting, Secret Redaction, Pytest, GitHub Actions CI/CD.

10. Technical Interview Preparation (Top 6 Selected Questions)

Q1: Why did you use mcp.server.lowlevel.Server instead of FastMCP?

FastMCP requires static function decorators (@mcp.tool()) declared at import time. An enterprise gateway discovers tools dynamically from upstream servers at runtime. Using the low-level Server API provides direct protocol callbacks (on_list_tools, on_call_tool) allowing dynamic schema aggregation and dispatch without generating stub code.

Q2: How do you guarantee stdio JSON-RPC wire purity in Claude Desktop integrations?

In stdio transport, stdout is reserved strictly for JSON-RPC 2.0 protocol frames. Any unexpected print() statements corrupt wire framing and disconnect the client. ToolBridge routes all internal logging, diagnostic banners, and error traces exclusively to stderr, and isolates child process pipes from host stdout.

Q3: Why is AST-based SQL inspection superior to regular expressions?

Regex-based SQL filters fail against whitespace variations, comment evasion (SELECT /*!50000...*/), or data-modifying Common Table Expressions (WITH x AS (DELETE...)). SQLSecurityGuard uses sqlparse to tokenize queries into an Abstract Syntax Tree, strictly enforcing single SELECT statements and walking the syntax tree to reject mutation clauses and dangerous functions.

Q4: How did you solve the AnyIO cancel scope cross-task exception?

Exiting an AsyncExitStack from a different asyncio task than the one that entered it throws 'Attempted to exit cancel scope in a different task'. In MCPClientManager, I assigned each upstream process to a dedicated, long-lived asyncio worker task that enters stdio_client, signals readiness via an asyncio.Event, holds the session, and awaits a stop event.

Q5: How does your 7-layer filesystem sandbox stop directory escapes?

FilesystemGuard rejects null bytes, relative drive letters, and traversal dots (.../). It resolves canonical paths via Path.resolve() and asserts canonical containment using resolved.relative_to(sandbox_root). Furthermore, it checks symlink targets, blocks Windows reserved DOS devices (CON, AUX, NUL), and executes atomic file replacements.

Q6: If asked to deploy ToolBridge to production, what architectural boundaries would you state?

A credible engineer acknowledges system boundaries: (1) The in-memory sliding-window rate limiter must be migrated to a distributed Redis cluster using a Token Bucket Lua script; (2) Credentials should be dynamically rotated via AWS Secrets Manager or HashiCorp Vault; and (3) Downstream transport should incorporate an SSE/WebSocket layer with mTLS termination.

Repository Link: https://github.com/Bhaveshbargat1/ToolBridge--MCP-Gateway | **License:** MIT | **Status:** Complete & Verified